



《软件测试》

单元测试

授课人：高利鹏

日期：2025年12月8日



目 录

CONTENTS

01 单元测试方法

02 单元测试内容

03 单元测试类型

04 断言

05 单元测试工具

06 小结

学习目标

- ◆ 掌握单元测试的方法和内容
- ◆ 了解单元测试的类型
- ◆ 掌握断言的使用
- ◆ 了解常用的单元测试工具

目 录

CONTENTS

01 单元测试方法

02 单元测试内容

03 单元测试类型

04 断言

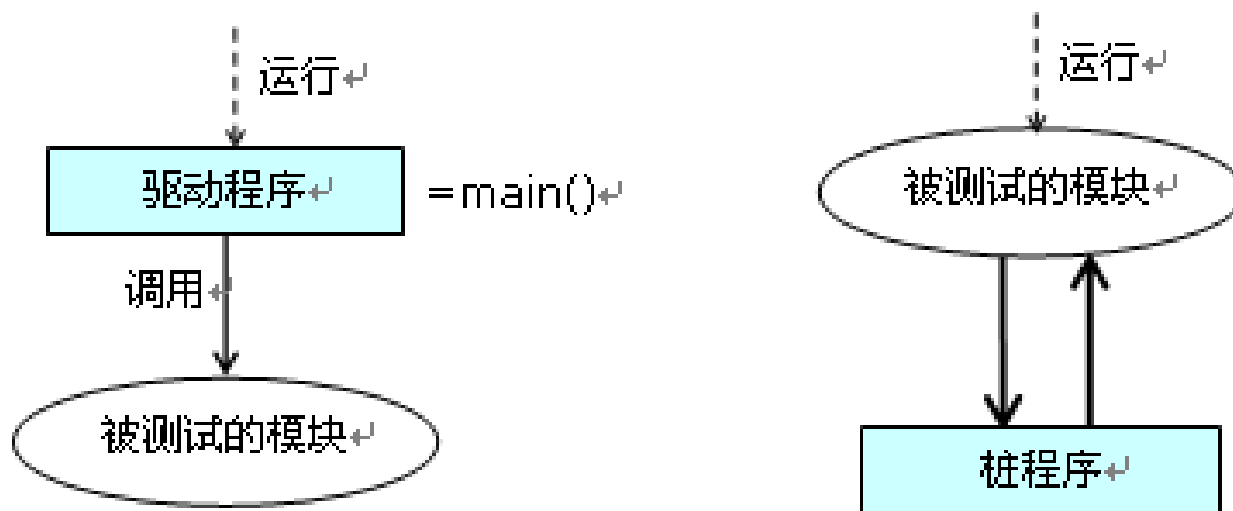
05 单元测试工具

06 小结

1 单元测试方法

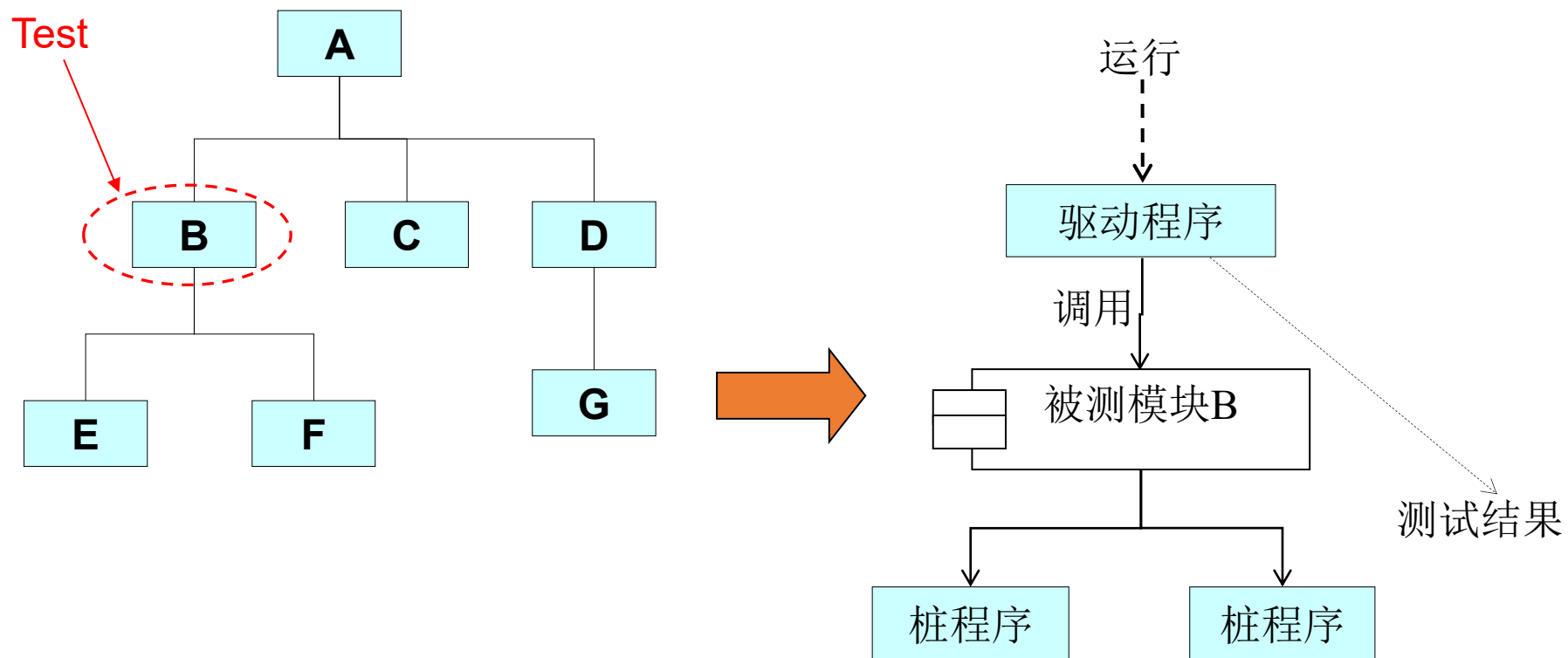
通常单元测试中的被测模块往往不是一个可以独立运行的程序，所以在执行单元测试阶段的动态测试时，应设置辅助模块，模拟被测模块与其它模块的联系，使得被测模块能正常运行，以达到测试目的。这个辅助模块可以分为两种：**驱动程序**和**桩程序**。

- **驱动程序 (driver)**，对底层或子层模块进行（单元或集成）测试时所编制的调用被测模块的程序，用以模拟被测模块的上级模块
- **桩程序 (stub)**，也有人称为存根程序，对顶层或上层模块进行测试时，所编制的替代下层模块的程序，用以模拟被测模块工作过程中所调用的模块。



1 单元测试方法

❖ 示例



目 录

CONTENTS

01 单元测试方法

02 单元测试内容

03 单元测试类型

04 断言

05 单元测试工具

06 小结

2 单元测试内容

(1) 模块接口测试

检查进出程序单元的数据流是否正确，这一过程必须要在其他测试之前进行。如果数据不能正确输入和输出，就谈不上进行其他测试。

模块接口测试项目主要包括：

- 调用所测模块时的输入参数与模块的形式参数在个数、属性、顺序上是否匹配
- 所测模块调用子模块时，它输入子模块的参数与子模块的形式参数在个数、属性、顺序上是否匹配
- 输出给标准函数的参数在个数、属性、顺序上是否匹配
- 全局变量的定义在各模块是否一致

2 单元测试内容

(2) 局部数据结构测试

最常见错误来源。主要测试项目包括：

- 检查不正确或者不一致的数据类型说明
- 使用尚未赋值或尚未初始化的变量
- 错误的初始值或者默认值
- 变量名拼写错误或书写错误
- 不一致的数据类型

2 单元测试内容

(3) 路径测试

单元测试中最基本测试类型，主要查找由于错误计算、不正确比较或不正常控制流而导致的错误。

常见的错误计算有：

- 运算的有限次序不正确或者误解了运算的有限次序
- 运算的方式错误（运算的对象彼此在类型上不相容）
- 算法错误
- 初始化不正确
- 运算精度不够
- 表达式的符号表示不正确

2 单元测试内容

(3) 路径测试（续）

常见的不正确比较和控制流错误有：

- 不同数据类型的比较
- 不正确的逻辑运算符或优先次序
- 因浮点运算精度问题而造成的两值比较不等
- “差1错”，即不正确地多循环或者少循环一次
- 错误的或不存在的循环终止条件
- 当遇到发散的迭代时不能终止循环
- 不适当地修改了循环变量

2 单元测试内容

(4) 错误处理测试

检查模块的错误处理功能是否有错误或缺陷。表明出错处理模块有错误或者有缺陷的情况有：

- 出错的描述难以理解
- 出错的描述不足以对错误定位和确定出错原因
- 显示的错误与实际错误不符
- 对错误条件的处理不正确
- 在对错误进行处理之前，错误条件已经引起系统的干预
- 如果出错情况不予考虑，那么检查恢复正常后模块可否正常工作

2 单元测试内容

(5) 边界测试

边界出错是常见的，应设计测试用例检查以下项目：

- 在n次循环的第0次，1次，n次是否有错
- 运算或者判断中取最大最小值时是否有错
- 数据流、控制流中刚好等于、大于或小于确定的比较值时出错的可能性

目 录

CONTENTS

01

单元测试方法

02

单元测试内容

03

单元测试类型

04

断言

05

单元测试工具

06

小结

3.1 单元测试类型



逻辑单元测试
(logic unit test)



集成单元测试
(integration unit test)



功能单元测试
(functional unit test)

- 逻辑单元测试 (logic unit test) 是针对单个方法进行代码正确性检查的测试。
- 集成单元测试 (integration unit test) 是针对组件之间的交互进行代码正确性检查的测试。
- 功能单元测试 (functional unit test) 将集成单元测试的边界进行了扩展，以确保正确地激发响应。

3.1 单元测试类型

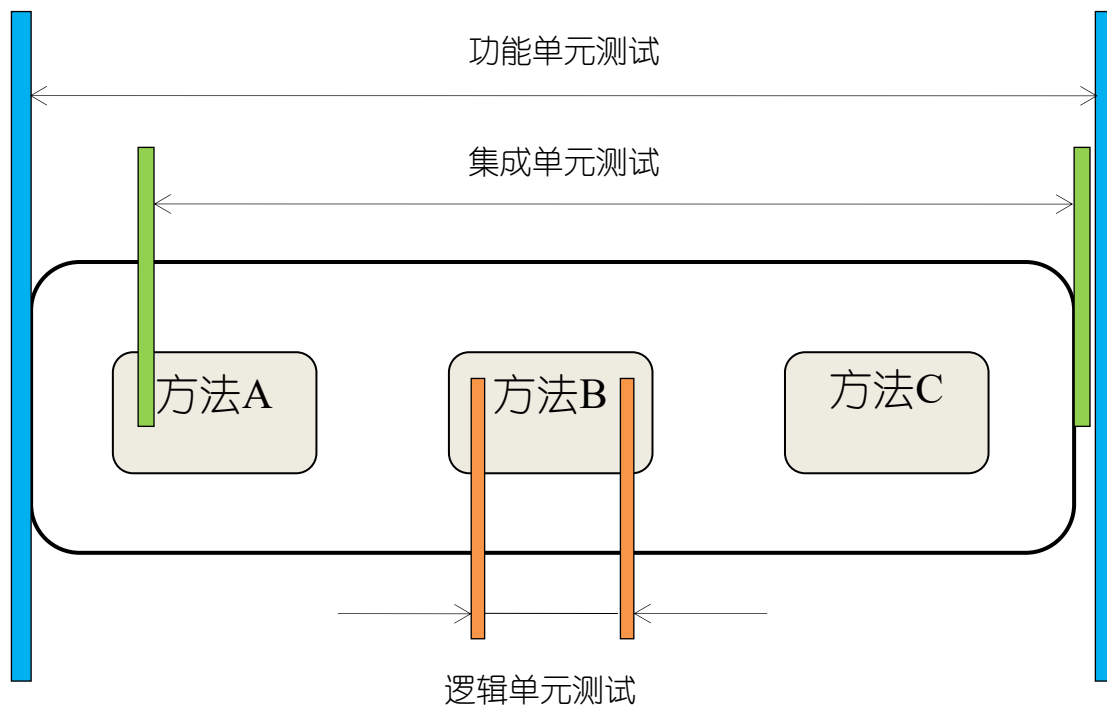


图1 单元测试类型及其边界

- 图1描述了三种单元测试类型之间的相互作用及其边界范围。

3.2 单元测试作用

执行单元测试的目的在于验证程序是否像预期的那样正确工作，并尽早地发现错误。除此之外，单元测试还会给开发人员带来一些益处：

- 编写单元测试可以帮助开发人员书写更高质量的代码。
- 编写单元测试可以使开发人员更有信心重构应用程序，去拥抱变化。

目 录

CONTENTS

01 单元测试方法

02 单元测试内容

03 单元测试类型

04 断言

05 单元测试工具

06 小结

4 断言

执行单元测试，是为了证明某段代码的行为确实和开发者所预期的结果一致。为了验证代码的行为是否和期望结果一致，就需要使用一些断言（assertion）。断言是一个简单的方法调用，用于判断某个语句是否为真。

```
Public void IsTrue(bool condition)
{
    If(!condition)
    {
        Abort();
    }
}
```

方法IsTrue检查给定的条件是否为真，如果条件非真，该断言将会失败。事实上，我们可以使用断言来检查所有的这类事物，比如，以下代码检查两个数字是否相等：

```
Public void AreEqual(int x, int y)
{
    IsTrue(x == y);
}
```

4 断言

有了以上两个断言，我们就可以编写具体测试脚本了。以下代码是一个从给定数组中查找最大值的函数，并对它进行测试。

```
public static int Largest(int [] list) {  
    int i_max = Int32.MaxValue;  
    for(int i_index =0; i_index <list.Length - 1; i_index ++)  
    {  
        If(list[i_index] > i_max)  
        {  
            i_max = list[i_index];  
        }  
    }  
    return i_max;  
}
```

4 断言

这是一个静态函数，用于查找list中的最大值。这里我们传递一个比较简单的没有重复元素的数组，数组中有3个元素，以下是对其进行测试的代码：

```
[Test]
public void TestLargest()
{
    int[] i_Arr = new int[3];
    i_Arr[0] = 6;
    i_Arr[1] = 7;
    i_Arr[2] = 5;
    AreEqual(7,Largest(i_Arr));
}
```

目 录

CONTENTS

01 单元测试方法

02 单元测试内容

03 单元测试类型

04 断言

05 单元测试工具

06 小结

5.1 JUnit概述

JUnit是一个Java编程语言的单元测试框架：

- JUnit 是一个开放的资源框架，用于编写和运行测试。
- 提供注释来识别测试方法。
- 提供断言来测试预期结果。
- JUnit 测试可以自动运行并且检查自身结果并提供即时反馈。所以也没有必要人工梳理测试结果的报告。
- JUnit 测试可以被组织为测试套件，包含测试用例，甚至其他的测试套件。
- JUnit 在一个条中显示进度。如果运行良好则是绿色；如果运行失败，则变成红色。

5.2 JUnit的基本用法

(1) 创建一个简单的类来作为测试用例

(2) 创建Test Case类

- 向测试类中添加测试方法方法；
- 在测试方法前加上注释@Test；
- 执行测试条件并应用JUnit中的API来检查。

(3) 创建Test Runner类

- 用JUnit的JUnitCore类中的runClasses方法来运行测试类的测试案例；
- 获取Result Object中运行的测试案例的结果；
- 获取Result Object的getFailures()中的失败结果；
- 获取Result Object的getSuccessful()中的成功结果。

5.3 JUnit中的API

- Assert: 断言方法的集合
- TestCase: 一个定义了运行 多重测试的固定装置
- TestResult: 集合了运行测试样例的所有结果
- TestSuite: 测试的集合

5.3 Junit中常用的断言

- **assertEquals断言**：这是应用非常广泛的一个断言，它的作用是比较实际的值和用户预期的值是否一样，assertEquals在JUnit中有很多不同的实现。
- **assertTrue与assertFalse断言**：assertTrue与assertFalse可以判断某个条件是真还是假，如果和预期的值相同则测试成功，否则将失败。
- **assertNull与assertNotNull断言**：可以验证所测试的对象是否为空或不为空，如果和预期的相同则测试成功，否则测试失败。
- **assertSame与assertNotSame断言**：assertSame和assertEquals不同，assertSame测试预期的值和实际的值是否为同一个参数(即判断是否为相同的引用)。assertNotSame则测试预期的值和实际的值是不为同一个参数。
- **fail断言**：“fail”断言能使测试立即失败，这种断言通常用于标记某个不应该被到达的分支。
- ...

5.4 JUnit中的注释

- `@Test` 这个注释说明依附在 JUnit 的 `public void` 方法可以作为一个测试案例。
- `@Before` 有些测试在运行前需要创造几个相似的对象。在 `public void` 方法加该注释是因为该方法需要在 test 方法前运行。
- `@After` 如果你将外部资源在 Before 方法中分配，那么你需要在测试运行后释放他们。在 `public void` 方法加该注释是因为该方法需要在 test 方法后运行。
- `@BeforeClass` 在 `public void` 方法加该注释是因为该方法需要在类中所有方法前运行。
- `@AfterClass` 它将会使方法在所有测试结束后执行。这个可以用来进行清理活动。
- `@Ignore` 这个注释是用来忽略有关不需要执行的测试的。

5.5 创建项目并书写测试代码

- 分别点击File -> New -> Java Project, 创建名为Triangle的项目。
- 将待测目标源码粘贴到src文件夹下。

Project name: Triangle

☒ Use default location

Location: D:\xingyu\Documents\eclipse-workspace\Triangle [Browse...](#)

JRE

☒ Use an execution environment JRE: JavaSE-1.8

☐ Use a project specific JRE: jre1.8.0_211

☐ Use default JRE (currently 'jre1.8.0_211') [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

5.5 创建项目并书写测试代码

- 在 Triangle.java 文件上右键选择 New -> JUnit Test Case。
- 将待测目标源码粘贴到 src 文件夹下
- 注意红色箭头处的选择，点击 Finish 即可。

☐ New JUnit 3 test ☒ New JUnit 4 test ☐ New JUnit Jupiter test

Source folder:

Package:

Name:

Superclass:

Which method stubs would you like to create?

☐ setUpBeforeClass() ☐ tearDownAfterClass()
☐ setUp() ☐ tearDown()
☐ constructor

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

Class under test:

目 录

CONTENTS

01 单元测试方法

02 单元测试内容

03 单元测试类型

04 断言

05 单元测试工具

06 小结

6 小结

这个知识点主要学习内容：

- 单元测试的方法
- 单元测试的内容
- 单元测试的类型
- 断言的用法
- 单元测试工具

习题

1. 什么是单元测试？单元测试主要应用于哪些方面？
2. 主要使用的单元测试工具有哪些？了解掌握测试工具的使用方法。